

Keshav Vinayak Jha

Detailed Academic Curriculum Vitae – Compiler and Systems Research

keshavvinayakjha@gmail.com | +91-7678442691 | keshavvinayak01.github.io
github.com/keshavvinayak01 | linkedin.com/in/keshavvinayakjha

Research Interests

Compiler and runtime systems for AI accelerators; GPU code generation and optimization; MLIR/LLVM; hardware–software co-design; GPU memory systems and virtual memory; computer-systems performance analysis.

Education

Indian Institute of Science (IISc)

M.Tech. (Research), Computer Science and Automation; GPA: 8.7/10

Bangalore, India

Aug. 2021 – Mar. 2024

- Advisor: [Prof. Arkaprava Basu](#); research collaboration with [Prof. Murali Annavaram](#).
- Research focus: DNN input-pipeline acceleration, GPU resource and memory management, and systems performance analysis.

University School of Information, Communication and Technology, GGSIPU

B.Tech., Electronics and Communication Engineering

New Delhi, India

2017 – 2021

Publication

Keshav Vinayak Jha, Shweta Pandey, Murali Annavaram, and Arkaprava Basu. “[HyCache: Hybrid Caching for Accelerating DNN Input Preprocessing Pipelines](#).” In *2025 USENIX Annual Technical Conference (USENIX ATC '25)*, pp. 433–448.

- Designed an automatic runtime that profiles a preprocessing pipeline and uses an integer linear program to choose which intermediate outputs to cache in memory or storage, balancing capacity, recomputation, and data-transfer cost.
- Evaluated image recognition, segmentation, object detection, and voice recognition pipelines across varied hardware configurations. HyCache improved raw pipeline throughput by $1.11\times$ – $10.1\times$ and end-to-end DNN training throughput by up to $1.67\times$.

Research and Compiler Experience

AMD – IREE and LLVM/AMDGPU

Senior Software Development Engineer

Remote

Aug. 2025 – Present

Develop production compiler infrastructure spanning PyTorch graph import, IREE’s structured and vector IRs, GPU dispatch formation, and LLVM AMDGPU instruction selection. Authored more than 50 merged public compiler contributions across IREE, IREE Turbine, torch-mlir, LLVM, ROCm libraries, and related projects during this period.

PyTorch-to-GPU attention compilation.

- Extended torch-mlir’s FX importer for higher-order operations: recursively imported child graphs for `while_loop` and represented FlexAttention score/mask functions and metadata in Torch IR ([torch-mlir #4338](#), [torch-mlir #4366](#)).
- Implemented lowering from PyTorch FlexAttention to IREE’s region-based `LinalgExt.OnlineAttentionOp`, including index expressions inside score regions and randomized numerical tests ([IREE #22768](#), [IREE #23292](#)).
- Carried online attention through dispatch formation, reshape fusion, encoding, dimension collapse, and GPU code generation; ported existing attention rewrites and enabled fusion through both accumulator and row-sum results ([IREE #24068](#), [IREE #24123](#), [IREE #24169](#)).
- Added grouped-query attention support across PyTorch metadata, IREE lowering, and the ROCm SDPA API by allowing independent query, key, and value head counts ([IREE #24313](#), [torch-mlir #4562](#), [ROCm #6186](#)).

Dispatch formation, fusion, and attention correctness.

- Repaired subtle correctness and code-generation failures involving fully masked attention rows, unsafe producer fusion, aligned shapes, suffix scans, unit-dimension folding, and small-shape GPU fallbacks ([IREE #24178](#), [IREE #24207](#), [IREE #23315](#), [IREE #24349](#), [IREE #24239](#)).
- Added compiler and benchmark integration for scaled dot-product attention in IREE Turbine and Fusilli, including post-decomposition replacement of PyTorch SDPA variants ([Turbine #1274](#), [Fusilli #263](#)).

Vector IR and GPU data movement.

- Collaborated on direct-to-LDS transfers and software-pipelined attention support in IREE's MI350X GPU pipeline.
- Added multi-batch gather vectorization to IREE's `transfer_gather` abstraction ([IREE #23552](#)).
- Designed and implemented most of an end-to-end `transfer_scatter` stack: operation semantics and verification, canonicalization, vector unrolling, and bufferization ([IREE #23666](#), [IREE #23696](#), [IREE #23704](#), [IREE #23719](#)). Layout analysis and nested-layout GPU distribution are ongoing ([IREE #23720](#)).
- Extended MLIR affine linearization/delinearization operations to vector indices and added vector canonicalizations that remove redundant element extraction/construction and transpose patterns ([LLVM #188369](#), [LLVM #160318](#), [LLVM #161841](#)).

Target-aware MLIR lowering and GPU performance.

- Implemented a target-aware vector `erf` approximation for ROCm using OCML's polynomial strategy, with fast-math guards and numerical validation ([IREE #21849](#)).
- Added an MLIR-to-LLVM path for AMDGPU's `fmed3` intrinsic and used it to lower clamp operations on architectures that support the instruction ([LLVM #159332](#), [LLVM #163520](#)).
- Extended compiler-driven GPU tuning with accumulator types, result types, and denormal-mode constraints ([SHARK #2839](#), [SHARK #2840](#)).
- Prototyped XOR-based LDS swizzle hints derived from distributed vector layouts. Evaluation over 220 f16/f32 matrix-multiplication shapes on MI300X exposed a datatype-dependent tradeoff: conflicts were removed broadly, but only the f32 cases showed a consistent runtime gain (ongoing [IREE #23778](#)).

LLVM AMDGPU backend correctness and infrastructure.

- Diagnose backend failures by tracing programs through LLVM IR, GlobalISel/SelectionDAG MIR, register allocation, and emitted AMDGPU instructions; add focused regression tests and hardware execution checks when instruction semantics are involved.
- Preserved known pointer bits when SelectionDAG demotes `sret` arguments, enabling AMDGPU buffer-address folding without a redundant add ([LLVM #203468](#)).
- Added true16 selection for scalar s16 `G_MERGE_VALUES`; repaired wide constant-remainder legalization when a high-multiply expansion is unavailable; and fixed a width-32 bitfield-extract wrong-code path caused by AMDGPU's five-bit width encoding ([LLVM #207999](#), [LLVM #210232](#), [LLVM #215457](#)).
- Fixed generic InstCombine infrastructure to rebuild type-changing PHI nodes during address-space-aware pointer replacement, preserving valid SSA typing ([LLVM #212429](#)).
- Ported AMDGPU GlobalISel divergence lowering to LLVM's New Pass Manager ([LLVM #216657](#)). Ongoing work adds GlobalISel lowering for preloaded kernel arguments, type-aware intrinsic feature checks, a New-Pass-Manager wrapper for the post-legalizer combiner, and CSE within that combiner ([LLVM #205049](#), [LLVM #205536](#), [LLVM #216274](#), [LLVM #217258](#)).

Cerebras Systems

Member of Technical Staff

Bangalore, India

Mar. 2024 – Jul. 2025

- Developed production compiler infrastructure for the Wafer-Scale Engine across Python and PyTorch frontends, torch-mlir/MLIR, PDL-based rewrites, tiered internal dialects, and low-level target IR.
- Implemented legality-preserving rewrites, canonicalizations, and lowering support across multiple abstraction layers, with emphasis on model coverage, compiler invariants, debuggability, regression testing, and maintainability.
- Upstreamed torch-mlir shape-inference and operator support for `aten.prod`, `randn.generator`, and `as_strided` under the public [keshavj-cerebras](#) account ([torch-mlir #4003](#), [torch-mlir #4062](#), [torch-mlir #4076](#)).

Indian Institute of Science – Computer Systems Lab

Graduate Researcher

Bangalore, India

Nov. 2022 – Mar. 2024

- Led the design and evaluation of HyCache. Built workload and hardware profilers, formulated the hybrid cache-placement problem as an integer linear program, and implemented the runtime on NVIDIA DALI.
- Studied concurrent DNN execution using NVIDIA Multi-Instance GPU (MIG) and streaming-multiprocessor partitioning, focusing on resource isolation, utilization, interference, and workload-sensitive partition sizes.
- Explored checkpointing for concurrent DNNs on one GPU, trading activation recomputation against memory footprint to improve model co-location under memory pressure.
- Built profiling and microbenchmark workflows to analyze GPU compute utilization, memory pressure, cache behavior, and scheduling interference.

Selected Academic Compiler and Systems Projects

MLIR Loop and Reduction Optimizations

Advanced Compilers, IISc

- Implemented an affine loop-interchange pass driven by an analytical locality model covering spatial reuse, temporal self-reuse, and temporal group reuse. The pass handles perfect loop nests and conservatively rejects unsupported control flow.
- Designed MLIR infrastructure and passes that recognize sequential CPU read-modify-write reductions and replace valid instances with atomic operations. Source: [course LLVM/MLIR fork](#).

Cross-Architecture CPU Performance Analysis

Computer Architecture, IISc

- Used SPEC CPU2017, `perf`, and hardware counters to compare AMD and Intel processors across IPC, branch prediction, L2 data-TLB behavior, cache behavior, and wasted work. Designed a branch-prediction stress microbenchmark that produced a 33% miss rate. [Report](#).

Application-Cooperative Linux Memory Ballooning

Operating Systems, IISc

- Modified Linux 5.11.5 to expose an application-facing ballooning API, monitor memory pressure, and signal registered processes to release memory. [Source and patch history](#).

CPU/GPU Matrix Multiplication Optimization

Computer Architecture, IISc

- Optimized GEMM using loop blocking, packing, AVX-512 vectorization, GPU scratchpad tiling, and hardware-counter-guided analysis. The final GTX 1650 kernel was $1017\times$ faster than the original naive single-threaded CPU baseline. [Report and artifacts](#).

Talks and Presentations

- HyCache: Hybrid Caching for Accelerating DNN Input Preprocessing Pipelines.** Paper presentation, USENIX ATC '25, Boston, July 2025. [Paper](#), [slides](#), and [video](#).
- Mitigating GPU Bank Conflicts with Swizzling.** Internal AMD compiler talk, July 2026. Explained LDS bank mapping, XOR-layout construction, IREE integration, and MI300X measurements. [Slides](#).
- Fast Algorithms for Convolutional Neural Networks.** IISc graduate seminar. Compared Winograd and FFT convolution in terms of arithmetic complexity, workspace, numerical behavior, and GPU implementation. [Slides](#).

Selected Graduate Coursework

- Computer Architecture** (10/10, highest in cohort): Prof. Arkaprava Basu
- Advanced Compilers** (9/10, highest in cohort): Prof. Uday Bondhugula
- Operating Systems** (8/10, highest in cohort): Prof. Arkaprava Basu
- Linear Algebra and Probability** (8/10): IISc

Honors and Service

- USENIX Professional Scholarship for ATC '25.
- Graduate Aptitude Test in Engineering (GATE) Computer Science 2021: All India Rank 390.
- Artifact Evaluation Committee Member, IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2027).

Technical Skills

Compiler and systems: LLVM, MLIR, IREE, torch-mlir, PyTorch, ROCm/AMDGPU, GlobalISel, SelectionDAG, Linux, `perf`, GPU profiling and benchmarking.

Programming: C++, C, Python, MLIR/LLVM TableGen, SQL, Git; working familiarity with JavaScript and web technologies.